

Smart Irrigation and Water Management System

1. Project Overview

The **Smart Irrigation and Water Management System** is an automated agricultural system that monitors soil moisture and controls water supply according to the actual requirement of the crop. The system uses a **soil moisture sensor, ESP8266 microcontroller, relay module, water pump, and water-level sensor**.

The soil moisture sensor continuously detects the moisture condition of the soil. When the soil becomes dry, the ESP8266 activates the water pump through a relay. Water is supplied to the plant through an irrigation pipe. Once the required moisture level is reached, the pump is automatically switched OFF.

The system can also monitor the water level in the storage tank and prevent the pump from operating when sufficient water is not available. This helps in reducing water wastage, minimizing human effort, and improving irrigation efficiency.

Objectives

- To automate the irrigation process.
- To monitor soil moisture continuously.
- To reduce unnecessary water consumption.
- To minimize manual intervention.
- To prevent overwatering and underwatering.
- To monitor the available water level.
- To improve agricultural water management.
- To develop a low-cost smart agriculture system.

2. Problem Statement

Traditional irrigation methods have several limitations that affect water efficiency and crop productivity.

Major Problems

1. Manual irrigation requires continuous human involvement.
2. Farmers may not know the actual moisture content of the soil.
3. Excessive irrigation results in water wastage.
4. Insufficient irrigation can affect plant growth.

5. Overwatering can cause root damage and soil nutrient loss.
6. Water pumps may operate unnecessarily.
7. Conventional systems do not respond automatically to soil conditions.
8. Lack of water-level monitoring can cause dry running of pumps.
9. Large agricultural areas require significant labour for irrigation.
10. Efficient utilization of limited water resources is difficult.

Problem Identification

The major problem is that conventional irrigation generally supplies water based on **time or manual judgment rather than actual soil moisture requirements**.

Therefore, an automatic sensor-based irrigation system is required.

3. Proposed Solution

The proposed system introduces an **IoT-based automatic irrigation mechanism**.

Proposed Method

The system consists of:

Soil Moisture Sensor → ESP8266 → Relay → Water Pump → Irrigation System

The soil moisture sensor measures the moisture content of the soil and sends the data to the ESP8266. The controller compares the measured value with a predefined threshold.

- **Dry soil → Pump ON**
- **Wet soil → Pump OFF**
- **Low tank water → Pump OFF + Alert**

Advantages of the Proposed Solution

- Automatic irrigation.
- Real-time soil monitoring.
- Reduced water wastage.
- Reduced labor requirements.
- Better crop water management.
- Protection against pump dry running.
- Low-cost implementation.

- Easy to expand for multiple plants.
 - Suitable for small farms and gardens.
 - Can be integrated with IoT monitoring.
-

4. System Architecture

The system architecture consists of sensing, processing, control, and irrigation sections.

The system architecture consists of a **soil moisture sensor, water-level sensor, ESP8266 microcontroller, relay module, water pump, buzzer, LED, and irrigation pipe**. The sensors collect soil and water-level information and send it to the ESP8266. The controller processes the sensor data and activates the relay to control the water pump. When the soil is dry and water is available, the pump turns ON and supplies water to the plants. When the soil reaches the required moisture level, the pump automatically turns OFF. The buzzer provides an alert when the water tank is empty.

Functional Blocks

1. Sensing Unit

- Soil moisture sensor
- Water-level sensor

2. Processing Unit

- ESP8266

3. Control Unit

- Relay module

4. Actuator

- DC water pump

5. Alert Unit

- Buzzer/LED

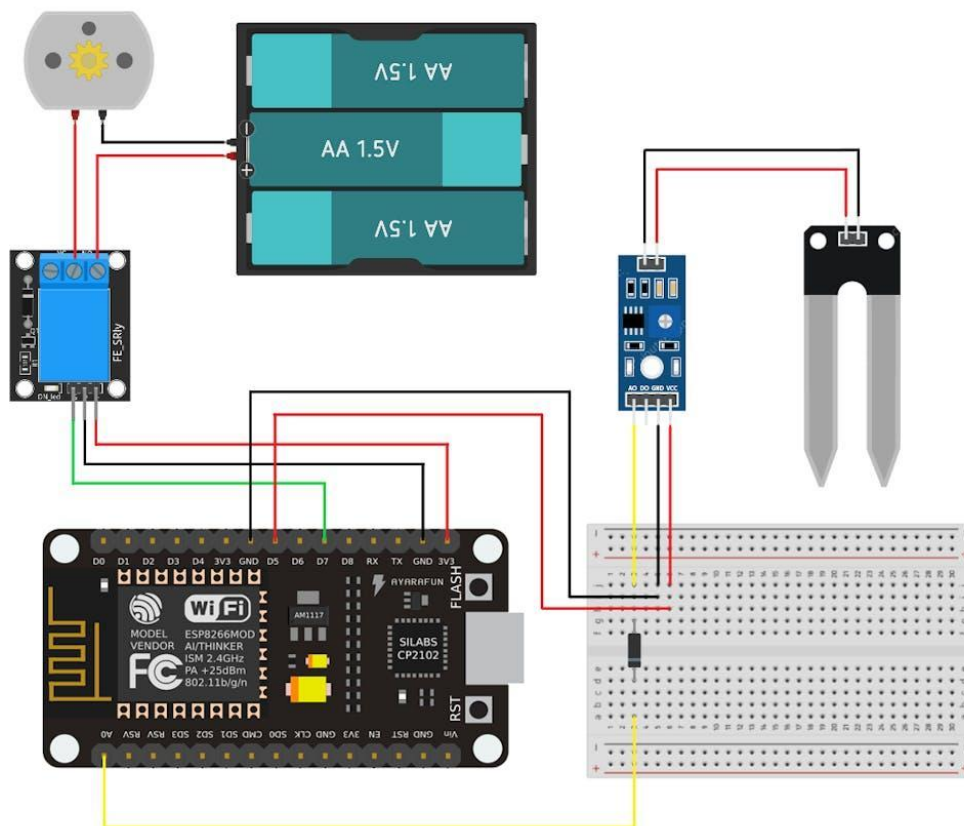
5. Circuit Table

S .No	Component	Pin	Connection
1	Soil Moisture Sensor	VCC	3.3V
2	Soil Moisture Sensor	GND	GND
3	Soil Moisture Sensor	AO	A0
4	Relay Module	VCC	Suitable supply
5	Relay Module	GND	GND
6	Relay Module	IN	D1
7	Water Level Sensor	VCC	3.3V
8	Water Level Sensor	GND	GND
9	Water Level Sensor	Signal	D2
10	Buzzer	+	D5
11	Buzzer	–	GND
12	LED	Anode	D6 through resistor
13	LED	Cathode	GND
14	Water Pump	+/-	External supply through relay

Important Note

The ESP8266 should **not directly power a high-current water pump**. The pump must use a suitable external power supply and be switched using the relay.

6. Circuit Design



When the sensor detects dry soil, the ESP8266 sends a control signal to the relay. The relay closes the pump circuit and the pump starts operating.

When the soil reaches the desired moisture level, the ESP8266 deactivates the relay and the pump stops.

Design Considerations

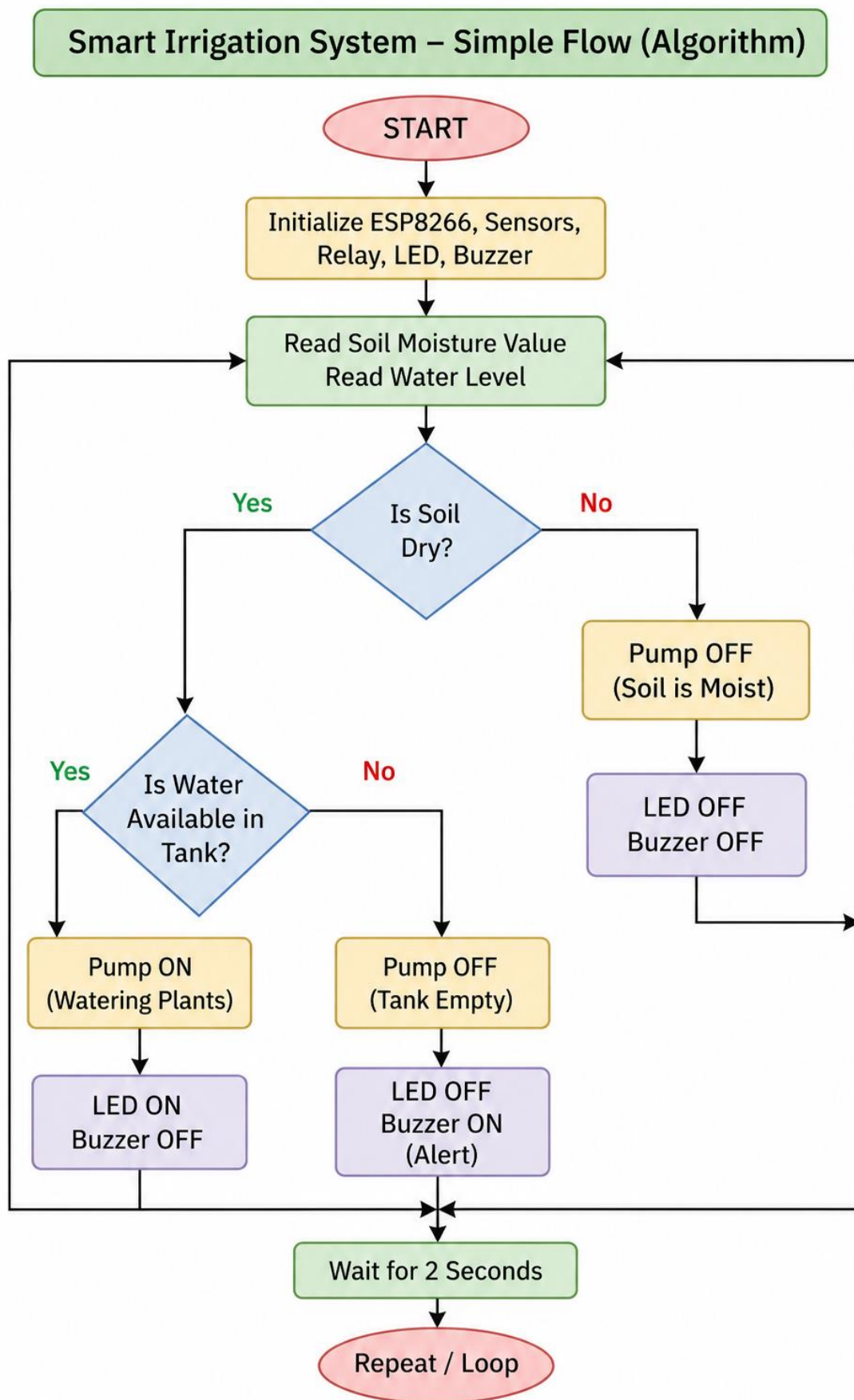
- Use a waterproof sensor for long-term outdoor operation.
- Use a suitable relay rating for the pump.
- Use an external supply for the pump.
- Keep sensor wiring protected from water.
- Calibrate the moisture sensor before operation.

7. Algorithm

Step-by-Step Algorithm

1. Start the system.
2. Initialize ESP8266.
3. Initialize soil moisture sensor.
4. Initialize water-level sensor.
5. Initialize relay, LED, and buzzer.
6. Read soil moisture value.
7. Read water tank level.
8. Compare soil moisture with the threshold.
9. If soil is sufficiently wet, keep the pump OFF.
10. If soil is dry, check the water tank.

o



8. Coding

```
#include <ESP8266WiFi.h>

#include "Adafruit_MQTT.h"
#include "Adafruit_MQTT_Client.h"
#include "DHT.h"

// =====

// WIFI DETAILS

// =====

#define WIFI_SSID    "Pradeep"
#define WIFI_PASSWORD "12345678"

// =====

// ADAFRUIT IO DETAILS

// =====

#define AIO_SERVER    "io.adafruit.com"
#define AIO_SERVERPORT 1883

#define AIO_USERNAME  "muthumkt"
#define AIO_KEY       "aio_Dpts45gXpVu4he3HHcwUXVBgOuKb"

// =====

// SENSOR PINS

// =====

#define SOIL_SENSOR A0
```



```
#define RELAY_PIN D5
```

```
#define DHT_PIN D7
```

```
#define DHT_TYPE DHT11
```

```
// =====
```

```
// DHT OBJECT
```

```
// =====
```

```
DHT dht(DHT_PIN, DHT_TYPE);
```

```
// =====
```

```
// WIFI + MQTT
```

```
// =====
```

```
WiFiClient client;
```

```
Adafruit_MQTT_Client mqtt(
```

```
  &client,
```

```
  AIO_SERVER,
```

```
  AIO_SERVERPORT,
```

```
  AIO_USERNAME,
```

```
  AIO_KEY
```

```
);
```

```
// =====
```

```
// ADAFRUIT IO FEEDS
```

```
// =====
```

```
Adafruit_MQTT_Publish soilFeed =
```

```
Adafruit_MQTT_Publish(  
    &mqtt,  
    AIO_USERNAME "/feeds/soil-moisture"  
);
```

```
Adafruit_MQTT_Publish temperatureFeed =
```

```
Adafruit_MQTT_Publish(  
    &mqtt,  
    AIO_USERNAME "/feeds/temperature"  
);
```

```
Adafruit_MQTT_Publish humidityFeed =
```

```
Adafruit_MQTT_Publish(  
    &mqtt,  
    AIO_USERNAME "/feeds/humidity"  
);
```

```
Adafruit_MQTT_Publish pumpFeed =
```

```
Adafruit_MQTT_Publish(  
    &mqtt,  
    AIO_USERNAME "/feeds/pump-status"  
);
```

```
// =====
```

```
// SOIL MOISTURE THRESHOLD
```

```
// =====
```

```
int dryThreshold = 700;
```

```
// =====  
// CONNECT TO WIFI  
// =====  
  
void connectWiFi()  
{  
  Serial.println();  
  Serial.print("Connecting to WiFi: ");  
  Serial.println(WIFI_SSID);  
  
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);  
  
  while (WiFi.status() != WL_CONNECTED)  
  {  
    delay(500);  
    Serial.print(".");  
  }  
  
  Serial.println();  
  Serial.println("WiFi Connected!");  
  
  Serial.print("IP Address: ");  
  Serial.println(WiFi.localIP());  
}  
  
// =====  
// CONNECT TO ADAFRUIT IO  
// =====
```

```
void connectMQTT()
{
    int8_t ret;

    if (mqtt.connected())
    {
        return;
    }

    Serial.println("Connecting to Adafruit IO...");

    while ((ret = mqtt.connect()) != 0)
    {
        Serial.print("MQTT Connection Failed: ");
        Serial.println(mqtt.connectErrorString(ret));

        mqtt.disconnect();

        delay(5000);

        Serial.println("Retrying...");
    }

    Serial.println("Adafruit IO Connected!");
}

// =====
// SETUP
```

```
// =====
```

```
void setup()
```

```
{
```

```
  Serial.begin(115200);
```

```
  delay(1000);
```

```
  Serial.println();
```

```
  Serial.println("=====");
```

```
  Serial.println(" SMART IRRIGATION SYSTEM");
```

```
  Serial.println("=====");
```

```
  // Pin configuration
```

```
  pinMode(SOIL_SENSOR, INPUT);
```

```
  pinMode(RELAY_PIN, OUTPUT);
```

```
  // Pump OFF at startup
```

```
  digitalWrite(RELAY_PIN, HIGH);
```

```
  // Start DHT11
```

```
  dht.begin();
```

```
  // Connect WiFi
```

```
  connectWiFi();
```

```
  // Connect Adafruit IO
```

```
  connectMQTT();
```

```
Serial.println("System Ready!");
Serial.println();
}

// =====
// MAIN LOOP
// =====

void loop()
{
  // Make sure WiFi is connected
  if (WiFi.status() != WL_CONNECTED)
  {
    connectWiFi();
  }

  // Make sure MQTT is connected
  connectMQTT();

  // =====
  // READ SOIL SENSOR
  // =====

  int soilValue = analogRead(SOIL_SENSOR);

  // Convert to percentage
  int moisturePercent = map(
    soilValue,
    1023,
```

```
0,  
0,  
100  
);
```

```
moisturePercent = constrain(  
    moisturePercent,  
    0,  
    100  
);
```

```
// =====  
// READ DHT11  
// =====
```

```
float temperature = dht.readTemperature();  
float humidity = dht.readHumidity();
```

```
// =====  
// PUMP CONTROL  
// =====
```

```
int pumpStatus;
```

```
if (soilValue > dryThreshold)  
{  
    // Soil is DRY  
    digitalWrite(RELAY_PIN, LOW);
```

```

    pumpStatus = 1;

    Serial.println("SOIL STATUS : DRY");
    Serial.println("PUMP STATUS : ON");
}
else
{
    // Soil is WET
    digitalWrite(RELAY_PIN, HIGH);

    pumpStatus = 0;

    Serial.println("SOIL STATUS : WET");
    Serial.println("PUMP STATUS : OFF");
}

// =====
// SERIAL MONITOR
// =====

Serial.println("-----");

Serial.print("Soil Raw Value : ");
Serial.println(soilValue);

Serial.print("Soil Moisture : ");
Serial.print(moisturePercent);
Serial.println("%");

```



```
if (isnan(temperature) || isnan(humidity))
{
    Serial.println("DHT11 ERROR!");
}
else
{
    Serial.print("Temperature   : ");
    Serial.print(temperature);
    Serial.println(" °C");

    Serial.print("Humidity     : ");
    Serial.print(humidity);
    Serial.println(" %");
}

Serial.println("-----");

// =====
// SEND SOIL DATA TO ADAFRUIT IO
// =====

if (soilFeed.publish(moisturePercent))
{
    Serial.println("Soil data sent to Adafruit IO");
}
else
{
    Serial.println("Failed to send soil data");
}
```

```
// =====  
// SEND TEMPERATURE  
// =====
```

```
if (!isnan(temperature))  
{  
  if (temperatureFeed.publish(temperature))  
  {  
    Serial.println("Temperature sent to Adafruit IO");  
  }  
  else  
  {  
    Serial.println("Failed to send temperature");  
  }  
}
```

```
// =====  
// SEND HUMIDITY  
// =====
```

```
if (!isnan(humidity))  
{  
  if (humidityFeed.publish(humidity))  
  {  
    Serial.println("Humidity sent to Adafruit IO");  
  }  
  else  
  {
```

```

        Serial.println("Failed to send humidity");
    }
}

// =====
// SEND PUMP STATUS
// =====

if (pumpFeed.publish(pumpStatus))
{
    Serial.println("Pump status sent to Adafruit IO");
}
else
{
    Serial.println("Failed to send pump status");
}

Serial.println();
Serial.println("Waiting 10 seconds...");
Serial.println();

// Wait before next reading
delay(10000);
}

```

9. System Working

The system starts when the ESP8266 is powered. The soil moisture sensor is inserted into the soil near the plant roots.

Working Sequence

- Step 1:** Soil moisture sensor measures soil moisture.
- Step 2:** Sensor sends the moisture value to ESP8266.
- Step 3:** ESP8266 compares the value with the preset threshold.
- Step 4:** If the soil is wet, the pump remains OFF.
- Step 5:** If the soil becomes dry, the controller checks the water tank.
- Step 6:** If sufficient water is available, the relay is activated.
- Step 7:** The pump starts and supplies water.
- Step 8:** Moisture increases in the soil.
- Step 9:** When the required moisture level is reached, the pump switches OFF.
- Step 10:** The controller continues monitoring the soil.

Operating Conditions

Soil Condition	Tank Condition	Pump	Alert
Wet	Water available	OFF	OFF
Dry	Water available	ON	OFF
Dry	Tank empty	OFF	ON
Wet	Tank empty	OFF	Optional

10. Bill of Materials

S. No	Component	Quantity	Purpose
1	ESP8266b Node MCU	1	Main controller
2	Soil Moisture Sensor	1	Soil moisture measurement
3	Relay Module	1	Pump control
4	DC Water Pump	1	Water supply
5	Water Level Sensor	1	Tank monitoring
6	Buzzer	1	Alert

S. No	Component	Quantity	Purpose
7	LED	1–2	Status indication
8	Resistor	1–2	LED protection
9	Breadboard	1	Circuit assembly
10	Jumper Wires	As required	Connections
11	Water Tank	1	Water storage
12	Irrigation Pipe	As required	Water distribution
13	External Power Supply	1	Pump power
14	USB Cable	1	ESP8266 programming

11. Prototype Implementation

The prototype is implemented using a small agricultural model consisting of a plant container, water tank, ESP8266 controller, sensors, relay, pump, and irrigation pipe.

Implementation Steps

1. Construct the prototype base.
2. Place the plant and soil inside the container.
3. Insert the soil moisture sensor into the soil.
4. Place the water-level sensor inside the tank.
5. Connect the sensors to ESP8266.
6. Connect the relay module to the ESP8266.
7. Connect the pump to the relay and external power supply.
8. Connect the irrigation pipe to the pump.
9. Upload the program to ESP8266.

10. Calibrate the soil moisture threshold.
11. Test the system with dry soil.
12. Test the system with wet soil.
13. Test the water tank empty condition.
14. Verify automatic pump operation.

12. Results & Performance Analysis

Expected Results

The developed system automatically controls irrigation based on soil moisture conditions. The pump operates only when irrigation is required, thereby reducing unnecessary water usage.

Parameter	Traditional System	Smart System
Irrigation	Manual	Automatic
Soil Monitoring	Manual	Sensor-based
Pump Control	Manual	Automatic
Water Wastage	High/Moderate	Reduced
Human Effort	High	Low
Overwatering	Possible	Minimized
Underwatering	Possible	Minimized
Tank Monitoring	Limited	Automatic
Response	Manual	Real-time
Scalability	Limited	High

Performance Analysis

- **Automatic operation:** Pump responds according to soil condition.

- **Water conservation:** Irrigation occurs only when required.
- **Reduced labor:** Continuous manual monitoring is not required.
- **Improved reliability:** Sensor-based control provides consistent operation.
- **Pump protection:** Water-level monitoring can prevent operation when the tank is empty.
- **Low-cost:** The prototype can be constructed using easily available components.
- **Scalability:** Multiple sensors and irrigation zones can be added.
- **IoT capability:** ESP8266 can be extended for remote monitoring

OUTPUT:

